

Lab 4

METHOD

I first began my design by drawing out a matrix on paper and reasoning about different ways I could go about the solution. The first idea I came up with was to simply loop through each array in the matrix. I knew this wouldn't be the final solution however, because the instructions said to ensure it was better than brute force. This approach was actually $O(m*n)$. Then I came up with a binary search approach, with did binary search on each row of the matrix, which would be $O(m*\log(n))$. My final solution was a divide-and-conquer approach which would be able to do it in $O(m+n)$.

APPROACH

I implemented two algorithms for this assignment to think about some of the pro's and con's to each choice.

The first uses a binary search helper function that gets called on each row of the matrix, this is how we get $O(m*\log(n))$, because we still loop through m rows, but the search on each row is done in $\log(n)$ time.

The second algorithm takes a divide-and-conquer approach starting at the top right index, and eliminating either a row or column at each step. This algorithm works for this type of matrix because it is sorted both by row and column. So at each step every number in the row to the left of the pointer is lower, and each number in the column under it is greater. So comparing the target to the current pointer means you can eliminate either a row or column in each pass. Which can be done in $O(m+n)$ time.

REASONING

For small matrices both the binary search algorithm and the dac algorithm are pretty similar in terms of actual time spent. However for large values of m and n , dac will be faster because it is linear.

REFLECTION

I used three different test cases,
 matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],
 [18,21,23,26,30]], target = 5
 Input: matrix = [[1,4,7,11,15],[2,5,8,12,19],[3,6,9,16,22],[10,13,14,17,24],
 [18,21,23,26,30]], target = 20
 and
 matrix = [[i + j * 1000 + 1 for i in range(1000)] for j in range(1000)]
 target = 42

The final case isn't strictly within the constraints of the problem, however it does show the reason one might choose different approaches based on input size.